GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 33 OF 42 | MORE PRACTICE CHAPTER TOPIC

More practice - Data structure and object design

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	6
EASY	MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Data structure and object design | 7 questions | tap a title to jump to it

E	1. Implement Batched Undo/Redo Layer	PracHub	2026-02-04
M	2. Implement Trie Insert and Exact-Word Search	PracHub	2026-02-20
M	3. Design Tic-Tac-Toe With K Players	PracHub	2026-02-12
M	4. Implement iterator merging lists with filtering	PracHub	2026-01-06
M	5. Restaurant Waitlist: join / delete / find_first_match	1Point3Acres	2026-02-01
M	6. Match people to questions using tags and priorities	PracHub	2025-12-27
M	7. Design server allocation for multi-type nodes	PracHub	2025-09-06

STUDY PATH: question 129 of 167 (more practice)

Priority score 0.96 | Data structure and object design | asked 258 days ago

EASY | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 1 of 7

Implement Batched Undo/Redo Layer

POSTED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 158/228**

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: LC 1472 Design Browser History (related: undo/redo stacks)

Posted on PracHub: 2026-02-04 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/implement-batched-undo-redo-layer

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value afte
r the batch string old_value; // value before the batc
h bool has_old_value; // whether the key existed befor
e the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

QUESTION

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op. Constraints
0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'),
    ('end',), ('snapshot',), ('undo',),
    ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color':
    'red'}]
```

Explanation: A single batch is committed, undone, and redone.

```
Input: [('begin',), ('set', 'a', '1'), ('end',),
    ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',),
    ('undo',), ('snapshot',), ('redo',),
    ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'},
    {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

APPROACH

Hold the open batch as a map of key -> (oldValue, newValue); endBatch pushes it onto the undo stack and clears the redo stack. undo applies the old values and moves the batch to the redo stack, redo reverses that. Handle an empty batch and repeated sets of one key in a batch (keep the first old value).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what undo does on an empty stack, whether an empty batch counts as an action, whether new edits after an undo clear the redo history (yes, normally), and how nested batches behave.

BRUTE FORCE TO SAY OUT LOUD

Snapshot the whole map after every batch and keep a list of snapshots. Simple but memory heavy.

OPTIMAL IDEA

Hold the open batch as a dict key to (old value, new value), recording the old value only the first time a key is touched in the batch. endBatch pushes it on the undo stack and clears the redo stack. undo pops, restores the old values (deleting keys that did not exist), and pushes onto redo. redo reapplies the new values. O(size of batch) each.

DRY-RUN THESE CASES

Set the same key twice in one batch, set a new key then undo (it must disappear), undo and redo with nothing available, a new batch after an undo (redo cleared), an empty batch.

LIKELY FOLLOW-UPS

Nested batches, bounded history, or undo of individual operations.

PITFALLS

Recording the old value on every set instead of only the first. Forgetting to clear redo on a new batch.

HOW TO THINK ABOUT IT

Undo and redo are two stacks of actions. The question is what an action is: a batch is one action, so record per batch only what is needed to reverse it (the old value of each key touched, and the new one), not a whole copy of the document. A new edit after an undo kills the redo history.

Basis: LeetCode 1472 (Design Browser History) for the stacks idea, plus transactional batching.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
undo, redo = [], []
def commit_batch():
    undo.append(copy.deepcopy(props))
    redo.clear()
def do_undo():
    redo.append(copy.deepcopy(props))
    props.clear()
    props.update(undo.pop())
def do_redo():
    undo.append(copy.deepcopy(props))
    props.clear()
    props.update(redo.pop())
```

TIME	SPACE
O(size of document) per batch	**O(size * batches)**

WHY IT IS SLOW

A one-key change copies the whole document, O(size) per batch and O(size * batches) memory. Storing only the old and new value of each changed key makes each batch O(keys changed).

OPTIMAL SOLUTION · Store only the changed keys per batch

TIME	SPACE
O(keys changed in the batch) per operation	**O(total changed keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

FOLLOW-UP 2 Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

FOLLOW-UP 3 Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

FOLLOW-UP 4 Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Batched Undo/Redo Layer
#
# You are implementing a simplified document layer for a
# design tool. The layer stores properties as a
# map<string, string>. Implement a class that supports the
# following operations: - beginBatch(): start a batch of
# edits. - setProperty(key, value): set or overwrite a
# property inside the current batch. - endBatch(): commit
# the batch as one undoable action. - undo(): revert the
# most recently committed batch. - redo(): reapply the
# most recently undone batch. Requirements: 1. A batch may
# update the same key multiple times. 2. undo() must
# restore the exact document state from before that batch.
# 3. redo() must restore the exact document state after
# that batch. 4. If a property did not exist before the
# batch, then undo() should remove that property. 5. If
# undo() has been called and then a new batch is
# committed, the redo history must be cleared. 6. Follow-
# up: make the history memory-efficient.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Limit the history to the last N batches.
# 2. Nested batches.
# 3. Undo of individual setProperty calls, not whole
# batches.
# 4. Limit the history to the last N batches.
#
# INTERVIEW NOTES
# Undo and redo are two stacks of actions. The question is
# what an action is: a batch is one action, so record per
# batch only what is needed to reverse it (the old value
# of each key touched, and the new one), not a whole copy
# of the document. A new edit after an undo kills the redo
# history.
#
# BRUTE FORCE (say it first): Snapshot the whole document
# after each batch
# Push a copy of the map on every commit; undo and redo
# move between snapshots. Obviously correct and a good
# oracle; it copies the whole document for a one-key
# change. Time O(size of document) per batch. Space O(size
# * batches).


from collections import deque
```

```


# HELPERS for the base question
class Change:
    """One key in one batch: value before and after."""

    def __init__(self, key, old, has_old, new):
        self.key, self.old, self.has_old, self.new = (
            key, old, has_old, new)


# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack.
# FOLLOW-UP 1: Limit the history to the last N batches.
# Answer: Keep the undo stack in a deque with maxlen N so
# the oldest batch is dropped automatically. Redo is
# unaffected.
# FOLLOW-UP 2: Nested batches.
# Answer: Keep a stack of open batches and merge a child
# into its parent on endBatch (keep the parent's old value
# if the key was already touched). Only the outermost
# endBatch commits an undoable action.
class Layer:
    def __init__(self, history=None):
        self.props = {}
        # FOLLOW-UP 1, 4: history=N keeps the last N
        # batches
        # committed batches; a deque with a maximum length
        # forgets the oldest ones when the history is
        # capped
        self.undo_stack = deque(maxlen=history)
        self.redo_stack = []
        # open batches (a stack, so batches can nest):
        # each is
        # a dict key -> Change, so a key appears only once
        self.open = []

    def begin_batch(self):
        self.open.append({})

    def set_property(self, key, value):
```

```
        if not self.open:
            # outside a batch: a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open[-1]
        if key in batch:
            # touched again: keep the ORIGINAL old value
            # and
            # only update the final value
            batch[key].new = value
        else:
            batch[key] = Change(key, self.props.get(key),
                                key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open.pop()
        if self.open:
            # FOLLOW-UP 2: nested batches fold into the
            # parent
            # a nested batch folds into its parent
            self.merge(self.open[-1], batch)
        elif batch:
            # only the outermost batch is one undoable
            # action;
            # a new action clears the redo history
            self.undo_stack.append(list(batch.values()))
            self.redo_stack.clear()

    @staticmethod
    def merge(parent, child):
        for key, change in child.items():
            if key in parent:
                parent[key].new = change.new
            else:
                parent[key] = change
```

```
    def undo(self):
        if not self.undo_stack:
            return False
        changes = self.undo_stack.pop()
        for c in changes:
            if c.has_old:
```

```
                self.props[c.key] = c.old
            else:
                # the key did not exist before: remove it
                self.props.pop(c.key, None)
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if not self.redo_stack:
            return False
        changes = self.redo_stack.pop()
        for c in changes:
            self.props[c.key] = c.new
        self.undo_stack.append(changes)
        return True


if __name__ == "__main__":
    layer = Layer()
    layer.begin_batch()
    layer.set_property("color", "red")
    layer.set_property("color", "blue")
    layer.set_property("size", "4")
    layer.end_batch()
    assert layer.props == {"color": "blue", "size": "4"}
    # each key is stored once, whatever the number of
    # edits
    assert len(layer.undo_stack[0]) == 2
    layer.undo()
    assert layer.props == {}
    layer.redo()
    assert layer.props == {"color": "blue", "size": "4"}
    layer.begin_batch()
    layer.set_property("color", "green")
    layer.end_batch()
    layer.undo()
    # the old value comes back, not the first value of the
    # batch
    assert layer.props["color"] == "blue"
    # a new batch clears the redo history
    layer.set_property("x", "1")
    assert not layer.redo()
    # nested batches commit as one action
    layer.begin_batch()
    layer.begin_batch()
```

```
    layer.set_property("y", "2")
    layer.end_batch()
    layer.set_property("z", "3")
    layer.end_batch()
    layer.undo()
    assert "y" not in layer.props
    assert "z" not in layer.props
    # history limited to the last 2 batches
    small = Layer(history=2)
    for i in range(4):
        small.set_property("k", str(i))
    assert len(small.undo_stack) == 2
    small.undo()
    small.undo()
    assert not small.undo() and small.props["k"] == "1"
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[list[str]]) -> list[dict[str
, str]]:
SAMPLE INPUT
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot
',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
```

Test cases

Input	Expected
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]	[{'color': 'red'}, {}, {'color': 'red'}]
[('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]	[{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]

STUDY PATH: question 130 of 167 (more practice)

Priority score 2.44 | Data structure and object design | asked 242 days ago

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 2 of 7

Implement Trie Insert and Exact-Word Search

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 107/228

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-02-20 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-trie-insert-and-exact-word-search

QUESTION

Problem

Implement a trie that supports **`insert(word)`** and **`search(word)`**. Insert adds a lowercase word. Search returns true only when the complete word was previously inserted; a stored word merely sharing the requested prefix is not a match. Process a sequence of operations and return the results of all searches.

Constraints & Assumptions

- There are at most 200,000 operations.
- Every word has length between 1 and 1,000.
- The total number of characters across all operations is at most 1,000,000.
- Words contain lowercase English letters.

Clarifications

- Inserting the same word repeatedly is allowed and has no additional effect.
- A node must distinguish an inserted word ending there from a prefix only.
- Return booleans in the same order as search operations appear.

Examples

```
operations = [insert("apple"), search("apple"), search("app"), insert("app"), search("app")] output = [true, false, true]
```

Hints

```
Represent prefixes Each edge consumes one character and each node represents the prefix along its path.
```

```
Mark complete words Reaching the final character is not enough unless the terminal node records an insertion.
```

APPROACH

Trie nodes with children and an end-of-word flag; search returns true only if the final node is marked as a word.

HOW TO THINK ABOUT IT

A trie needs a flag that marks 'a word ends here'; without it search('app') would be true whenever 'apple' was inserted. Walking a prefix and checking the flag are the two operations: search checks the flag, startsWith does not.

Basis: LeetCode 208, Implement Trie (Prefix Tree).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
words = []
def insert(w):
    words.append(w)
def search(w):
    return w in words
def starts_with(p):
    for word in words:
        if word.startswith(p):
            return True
    return False
```

TIME	SPACE
O(W * L) per operation	O(total characters)

WHY IT IS SLOW

Each call scans every stored word, O(W * L). A trie with an end-of-word flag makes every call O(L).

OPTIMAL SOLUTION · Trie of nodes with an end-of-word flag

TIME	SPACE
O(L) per operation	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

FOLLOW-UP 2 Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. **Cost:** O(L) time and O(L) space for the path stack.

FOLLOW-UP 3 Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. **Cost:** O(L) per operation with dictionary children; memory O(total characters).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Trie Insert and Exact-Word
# Search
#
# ## Problem Implement a trie that supports insert(word)
# and search(word). Insert adds a lowercase word. Search
# returns true only when the complete word was previously
# inserted; a stored word merely sharing the requested
# prefix is not a match. Process a sequence of operations
# and return the results of all searches. ### Constraints
# & Assumptions - There are at most 200,000 operations. -
# Every word has length between 1 and 1,000. - The total
# number of characters across all operations is at most
# 1,000,000. - Words contain lowercase English letters.
# ### Clarifications - Inserting the same word repeatedly
# is allowed and has no additional effect. - A node must
# distinguish an inserted word ending there from a prefix
# only. - Return booleans in the same order as search
# operations appear.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support '.' as a wildcard in search (LeetCode 211).
# 2. Delete a word.
# 3. Unicode or large alphabets.
#
# INTERVIEW NOTES
# A trie needs a flag that marks 'a word ends here';
# without it search('app') would be true whenever 'apple'
# was inserted. Walking a prefix and checking the flag are
# the two operations: search checks the flag, startsWith
# does not.
#
# BRUTE FORCE (say it first): Keep a list of words and
# scan
# Store the words in a list; search compares equality with
# each, startsWith tests the prefix on each. Correct, with
# every call linear in the dictionary size. Time O(W * L)
# per operation. Space O(total characters).


# BRUTE FORCE (say it first): Keep a list of words and
# scan | time O(W * L) per operation
class TrieBrute:
    def __init__(self):
        self.words = []
```

```
    def insert(self, word):
        self.words.append(word)
```

```
    def search(self, word):
        return word in self.words

    def startsWith(self, prefix):
        return any(
            w.startswith(prefix) for w in self.words
        )


# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        self.end = False


# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
# FOLLOW-UP 1: Support '.' as a wildcard in search
# (LeetCode 211).
# Answer: On '.', try every child at that level and
# recurse; other characters follow the single matching
# child. Worst case O(26^L) per query on adversarial
# patterns, so mention the bound.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters; None if the path breaks
        node = self.root
        for ch in text:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
```

```
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
        node.end = True

    def search(self, word):
        # the whole word must have been inserted
        node = self.walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self.walk(prefix) is not None

    # FOLLOW-UP 1: '.' matches any one letter
    def search_wild(self, pattern):
        # '.' matches any one letter (LeetCode 211)
        def go(node, i):
            if i == len(pattern):
                return node.end
            ch = pattern[i]
            if ch == ".":
                # try every child at this level
                kids = node.children.values()
                return any(go(c, i + 1) for c in kids)
            child = node.children.get(ch)
            return child is not None and go(child, i + 1)

        return go(self.root, 0)

    # FOLLOW-UP 2: delete a word
```

```
    def delete(self, word):
        # unmark the word, then prune nodes nothing needs
        path = [self.root]
        for ch in word:
            nxt = path[-1].children.get(ch)
            if nxt is None:
                return False
            path.append(nxt)
        if not path[-1].end:
            return False
        path[-1].end = False
        # walk back up removing empty, non-word nodes
        for i in range(len(word), 0, -1):
            node = path[i]
            if node.end or node.children:
```

```
                break
            del path[i - 1].children[word[i - 1]]
        return True


# HELPERS for the checks below
def run(operations):
    # ["insert", w] or ["search", w] -> results of the
    # searches
    trie, out = Trie(), []
    for op, word in operations:
        if op == "insert":
            trie.insert(word)
        else:
            out.append(trie.search(word))
    return out


if __name__ == "__main__":
    ops = [("insert", "apple"), ("search", "apple"),
           ("search", "app"), ("insert", "app"),
           ("search", "app")]
    assert run(ops) == [True, False, True]
    t = Trie()
    t.insert("apple")
    # a shared prefix is not a match for search
    assert not t.search("app") and t.starts_with("app")
    assert not t.starts_with("b")
    t.insert("bad")
    t.insert("dad")
    assert t.search_wild("b.d") and t.search_wild(".ad")
    assert not t.search_wild("b..d")
    assert t.search_wild("...")
    # delete keeps shorter words that share the path
    t.insert("app")
    assert t.delete("apple")
    assert t.search("app") and not t.search("apple")
    assert not t.delete("apple")
    assert t.delete("app") and not t.starts_with("a")
```

FUNCTION AND TEST CASES

Signature

```
def process_trie_operations(operations: list[list[str]]) -
> list[bool]:
```

Sample call

```
process_trie_operations([['insert', 'apple'], ['search', '
apple'], ['search', 'app'], ['insert', 'app'], ['search',
'app']])
```

Test cases

Input	Expected
`([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['search', 'app']],)`	`[True, False, True]`
`([],)`	`[]`

STUDY PATH: question 131 of 167 (more practice)

Priority score 2.22 | Data structure and object design | asked 250 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 3 of 7

Design Tic-Tac-Toe With K Players

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 153/228

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: the interview site's own solution

LeetCode match: LC 348 Design Tic-Tac-Toe

Posted on PracHub: 2026-02-12 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/design-tic-tac-toe-with-k-players

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.

QUESTION

- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2,
    1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue',
    'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

APPROACH

Per-player row, column, diagonal and anti-diagonal counters; a player wins when any counter reaches n. O(1) per move.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a win needs a full row, column or diagonal (n in a row) or only m in a row, how invalid moves are reported, what happens on a full board, and the API (move(row, col, player) returns winner or none).

BRUTE FORCE TO SAY OUT LOUD

After each move scan the row, column and both diagonals of the board. O(n) per move.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Keep counters per player for each row, each column, the main diagonal and the anti-diagonal. A move increments up to four counters; a player wins when any of their counters reaches n. O(1) per move, O(k * n) memory using a dict of counters. Reject moves on occupied cells.

DRY-RUN THESE CASES

A diagonal win, an anti-diagonal win, a row win by the last player, a draw, a move on an occupied cell, a move after the game has ended.

LIKELY FOLLOW-UPS

Win length m smaller than n (scan four directions from the last move for up to m cells), undo a move, or an unbounded sparse board.

PITFALLS

Counting an occupied cell twice. Not ending the game after a win.

HOW TO THINK ABOUT IT

Do not look at the board after each move. A win needs n marks of the same player on one line, so keep a counter per player and per line (each row, each column, the two diagonals) and bump the at-most-four lines that the move touches. The move wins if any of those counters reaches n.

Basis: LeetCode 348, Design Tic-Tac-Toe, generalized to k players. Assumed rule (confirm it): a player wins by filling a whole row, a whole column or a whole diagonal of the n x n board.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def move(player, r, c):
    board[r][c] = player
    lines = [row(r), column(c)]
    if r == c:
        lines.append(main_diagonal())
    if r + c == n - 1:
        lines.append(anti_diagonal())
    for line in lines:
        if line.count(player) == n:
            return "WIN"
    return "CONTINUE"
```

TIME	SPACE
O(n) per move	O(n^2)

WHY IT IS SLOW

Each move rescans up to four lines, O(n). Keeping a counter per player per row, column and diagonal makes each move O(1).

OPTIMAL SOLUTION · Counters per player and per line

TIME	SPACE
O(1) per move	O(k * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(m) per move.

FOLLOW-UP 2 Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

FOLLOW-UP 3 The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. **Cost:** O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design Tic-Tac-Toe With K Players
#
# Design and implement a Tic-Tac-Toe game engine. You are
# given: - k players, each with a unique player ID or
# mark. - An n x n board, initially empty. - Players take
# turns placing their own mark on an empty cell. Unlike
# classic Tic-Tac-Toe, a player wins as soon as they form
# **3 consecutive marks** in a straight line, regardless
# of how large n is. A straight line may be horizontal,
# vertical, diagonal down-right, or diagonal down-left.
# Implement a class or set of functions that supports
# making moves and returns the game status after each
# move, such as: - "Game Over" if a move is attempted
# after the game has ended. - "<player> won" when the
# latest move causes that player to have 3 consecutive
# marks. - A non-terminal status such as "Continue" if the
# game should keep going. Your implementation should
# handle invalid moves, such as placing a mark outside the
# board or on an occupied cell.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Win with m in a row, not the full line.
# 2. Undo the last move.
# 3. The board is huge or unbounded.
#
# INTERVIEW NOTES
```

```python
# Do not look at the board after each move. A win needs n
# marks of the same player on one line, so keep a counter
# per player and per line (each row, each column, the two
# diagonals) and bump the at-most-four lines that the move
# touches. The move wins if any of those counters reaches
# n.
#
# BRUTE FORCE (say it first): Scan the lines through the
# last move
# Store the board and, after each move, check the move's
# row, column and (if on one) the diagonals for n equal
# marks. Simple and fast enough for small n, but every
# move rescans up to four lines. Time O(n) per move. Space
# O(n^2).


# BRUTE FORCE (say it first): Scan the lines through the
# last move | time O(n) per move
class TicTacToeBrute:
```

```python
    def __init__(self, n):
        self.n, self.board = n, [
            [0] * n for _ in range(n)
        ]

    def move(self, r, c, player):
        if self.board[r][c]:
            return -1
        self.board[r][c] = player
        n, b = self.n, self.board
        if all(
            b[r][j] == player for j in range(n)
        ) or all(b[i][c] == player for i in range(n)):
            return player
        if r == c and all(
            b[i][i] == player for i in range(n)
        ):
            return player
        if r + c == n - 1 and all(
            b[i][n - 1 - i] == player for i in range(n)
        ):
            return player
        return 0


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BASE QUESTION (optimal): Counters per player and per
# line
# Idea: rows[p][r], cols[p][c], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n.
# FOLLOW-UP 1: Win with m in a row, not the full line.
# Answer: Counters per line no longer work, because a line
# can hold a run anywhere. After each move look along the
# four directions from that cell, counting same-player
# neighbours up to m in both directions. O(m) per move.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        # FOLLOW-UP 3: a huge or unbounded board
        # only occupied cells are stored (works for huge
```

```python
        # boards)
        self.cells = {}
        self.history = []
        self.winner = None

    def turn(self):
        # players go 1, 2, ..., k, then 1 again
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        # an invalid move changes nothing
        if not 1 <= player <= self.k:
            return "Invalid move"
        if player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n):
            return "Invalid move"
        if (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        # only the four lines through the new mark can win
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc, player)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.winner = player
                return f"{player} won"
        return "Continue"

    # FOLLOW-UP 2: undo the last move
    def undo(self):
        # take the last move back (also reopens a finished
```

```python
        # game)
        if not self.history:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True


# HELPERS for the checks below
def process(n, k, moves):
    game = TicTacToe(n, k)
    return [game.move(*m) for m in moves]


if __name__ == "__main__":
    moves = [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1],
             [1, 0, 2], [2, 2, 2]]
    assert process(3, 2, moves) == [
        "Continue", "Continue", "Continue", "Continue",
        "1 won", "Game Over"]
    # invalid moves do not use up the turn
    got = process(3, 2, [[2, 0, 0], [1, 5, 5], [1, 0, 0],
                         [2, 0, 0], [3, 1, 1], [2, 1, 1]])
    bad = "Invalid move"
    ok = "Continue"
    assert got == [bad, bad, ok, bad, bad, ok]
    g = TicTacToe(3, 2)
    for m in moves[:5]:
        g.move(*m)
    assert g.winner == 1 and g.undo() and g.winner is None
    assert g.move(1, 0, 2) == "1 won"
    # a huge board costs nothing: only taken cells are
    # stored
    big = TicTacToe(10 ** 9, 2)
```

```
    assert big.move(1, 10 ** 9 - 1, 5) == "Continue"
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, k: int, moves: list[list[int]]) -> li
st[str]:
```

Sample call

```
solution(3, 2, [])
```

Test cases

Input	Expected
`(3, 2, [])`	`[]`
`(5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])`	`['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']`

ALSO REPORTED · 1 more

Tic-Tac-Toe System with K Players and 3-in-a-Row Win | 1Point3Acres | phone-screen, tech-screen | asked 2026-05-18 | Single report

https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.
Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.
Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

Preparation: Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY PATH: question 132 of 167 (more practice)

Priority score 1.66 | Data structure and object design | asked 287 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 4 of 7

Implement iterator merging lists with filtering

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 155/228

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #281 Zigzag Iterator

LeetCode match: LC 281 Zigzag Iterator (+ filter/dedupe) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/implement-iterator-merging-lists-with-filtering

QUESTION

Overview: This question evaluates iterator/generator design, sequence merging, stateful filtering and deduplication, checking a candidate's ability to maintain input ordering while excluding blocked IDs and preventing duplicate yields.

APPROACH

Keep one pointer per list and a lookahead 'next valid' value for hasNext/next; skip filtered and duplicate values; or merge with a heap if sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how the lists are merged: round robin (like LeetCode 281 Zigzag Iterator), one after the other, or by sorted value; what is filtered (a blocked set of ids); whether duplicates must be removed across lists; and what next() does when the iterator is exhausted.

BRUTE FORCE TO SAY OUT LOUD

Build one merged list up front with the filter applied. Simple but not lazy and not memory friendly.

OPTIMAL IDEA

A lazy iterator with one pointer per list and a precomputed next valid value (lookahead). hasNext checks the lookahead; next returns it and then advances to the following valid element: skip blocked ids and anything already in a seen set. For sorted merges use a heap of (value, list index, position). Each element is touched once, O(total) overall.

DRY-RUN THESE CASES

Empty lists, all values blocked, duplicates across lists, calling next after the end, a list that becomes empty mid-way.

LIKELY FOLLOW-UPS

Make the iterator re-iterable, support remove, or bound memory for the seen set when ids are unbounded (Bloom filter).

PITFALLS

Computing the lookahead lazily at the wrong moment so hasNext skips or repeats a value.

INTERVIEW-DAY PLAN

Empty lists, all values blocked, duplicates across lists, calling next after the end, a list that becomes empty mid-way.

LIKELY FOLLOW-UPS

Make the iterator re-iterable, support remove, or bound memory for the seen set when ids are unbounded (Bloom filter).

PITFALLS

Computing the lookahead lazily at the wrong moment so hasNext skips or repeats a value.

HOW TO THINK ABOUT IT

An iterator does its work lazily: hasNext() must be able to answer without consuming. So keep a lookahead, the next valid value already computed, and advance to the one after it when next() is called. Filtering and de-duplication live in the 'find the next valid value' step so both hasNext and next stay trivial.

Basis: LeetCode 281, Zigzag Iterator, with a blocked-id filter and no duplicate output. Assumed rules (confirm them): lists are read round robin; ids in the blocked set are skipped; each id is yielded at most once.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merged_iterator(lists, blocked):
    seen = set()
    for x in round_robin(lists):
        if x in blocked or x in seen:
            continue
        seen.add(x)
        yield x
```

TIME	SPACE
O(total)	O(total)

WHY IT IS SLOW

It reads everything up front and keeps it all in memory, so it is not lazy and never works on a long or unbounded source. A lookahead value per iterator computes only the next valid item.

OPTIMAL SOLUTION · Lazy iterator with one pointer per list and a lookahead

TIME	SPACE
O(1) amortized per value	O(number of lists + distinct ids yielded)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

FOLLOW-UP 2 Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

FOLLOW-UP 3 The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. **Cost:** O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false suppressions.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement iterator merging lists with
# filtering
#
# This question evaluates iterator/generator design,
# sequence merging, stateful filtering and deduplication,
# checking a candidate's ability to maintain input
# ordering while excluding blocked IDs and preventing
# duplicate yields.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are sorted and the output must stay sorted.
# 2. Read the lists one after another, not interleaved.
# 3. The seen set grows without bound.
#
# INTERVIEW NOTES
# An iterator does its work lazily: hasNext() must be able
# to answer without consuming. So keep a lookahead, the
# next valid value already computed, and advance to the
# one after it when next() is called. Filtering and de-
# duplication live in the 'find the next valid value' step
# so both hasNext and next stay trivial.
#
# BRUTE FORCE (say it first): Build the full merged,
# filtered list first
# Interleave the lists, drop blocked and repeated ids, and
# iterate over the resulting list. Correct and a good
# oracle, but it is not lazy: it reads everything up front
# and holds it all in memory. Time O(total). Space
# O(total).


import heapq
from itertools import chain


# BRUTE FORCE (say it first): Build the full merged,
# filtered list first | time O(total)
def merged_filtered_brute(lists, blocked):
    out, seen = [], set()
    for step in range(
        max((len(l) for l in lists), default=0)
    ):
        for l in lists:
            if (
                step < len(l)
                and l[step] not in blocked
                and l[step] not in seen
            ):
                seen.add(l[step])
                out.append(l[step])
    return out


# BASE QUESTION (optimal): Lazy iterator with one pointer
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
class FilteredZigzag:
    """Take one item from each list in turn; skip blocked
    ids and ids already returned. Has hasNext / next."""

    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.pos = [0] * len(lists)
        self.turn = 0
        self.seen = set()
        # one item of lookahead makes hasNext easy
        self.ahead = None
        self.advance()

    def advance(self):
        self.ahead = None
        misses = 0
        # stop when every list had nothing left in a full
        # round
        while misses < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.pos[i] >= len(self.lists[i]):
                misses += 1
                continue
            value = self.lists[i][self.pos[i]]
            self.pos[i] += 1
            misses = 0
            blocked = value in self.blocked
            skip = blocked or value in self.seen
            if not skip:
                self.ahead = value
                return

    def has_next(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value


# FOLLOW-UP 1: The lists are sorted and the output must
# stay sorted.
# Answer: Replace the round robin with a k-way merge on a
# heap (heapq.merge), and drop a value equal to the
# previous output instead of keeping a seen set: memory
# O(k).
```

```
def merge_sorted(lists, blocked=()):
    # sorted lists, sorted output: a k-way merge; equal
    # values
    # come out together, so remember only the previous one
    blocked, last = set(blocked), object()
    for value in heapq.merge(*lists):
        if value in blocked or value == last:
            continue
        last = value
        yield value


# FOLLOW-UP 2: Read the lists one after another, not
# interleaved.
# Answer: Chain the lists with a generator; the same
# blocked/seen filtering applies. Generators give hasNext-
# style behaviour via a one-item lookahead.
def read_in_turn(lists, blocked=()):
    # one list after another, with the same filtering
    blocked, seen = set(blocked), set()
    for value in chain.from_iterable(lists):
        if value not in blocked and value not in seen:
            seen.add(value)
            yield value
```

```
if __name__ == "__main__":
    lists = [[1, 2, 3], [2, 4], [5]]
    it = FilteredZigzag(lists, blocked=[4])
    out = []
    while it.has_next():
        out.append(it.next())
    # 1, 2, 5, then 2 (seen), 3, 4 (blocked)
    assert out == [1, 2, 5, 3]
    empty = FilteredZigzag([[], []])
    assert not empty.has_next()
    assert list(merge_sorted([[1, 3, 5], [1, 2, 5]],
                             blocked=[3])) == [1, 2, 5]
    chained = read_in_turn([[1, 2], [2, 3]], [1])
    assert list(chained) == [2, 3]
```

FUNCTION AND TEST CASES

Sample call

```
solution([3, 1, 2, 3], [2, 4, 1, 5], {4})
```

Test cases

Input	Expected
`([3, 1, 2, 3], [2, 4, 1, 5], {4})`	`[3, 1, 2, 5]`
`([], [1, 1], set())`	`[1]`

STUDY PATH: question 133 of 167 (more practice)

Priority score 1.5 | Data structure and object design | asked 261 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 5 of 7

Restaurant Waitlist: join / delete / find_first_match

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 216/228

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

QUESTION

Onsite coding round: design and implement a restaurant waitlist data structure supporting join, delete, and find_first_match(table_size). The find call returns the earliest-joined user whose party size fits the given table.

Implement a class with these methods

join(user) — append user (with their party_size) to the waitlist.
delete(user) — remove user from the waitlist (could be anywhere in the queue).
find_first_match(table_size: int) — return the earliest user still on the waitlist whose party_size <= table_size. Do not remove on find.

Implicit constraints

Order of joins matters (FIFO within those that match).
Users may be deleted out of order, so a plain array/linked list with O(1) join cannot give O(1) delete and find together.

EXAMPLES

```
join(Alice, size=2)
join(Bob, size=4)
join(Carol, size=6)
find_first_match(3) -> Alice   // only Alice fits
find_first_match(5) -> Alice   // Alice still earliest
    matching
delete(Alice)
find_first_match(5) -> Bob
find_first_match(2) -> null     // no one fits
```

APPROACH

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

HOW TO THINK ABOUT IT

Join needs the order of arrival, delete can hit anyone, and find must return the earliest person whose party fits the table. Keep one queue per party size and a sequence number per person: find looks at the front of every queue for sizes up to the table size and takes the smallest sequence number. Deletions are marked and skipped lazily when a queue front is read.

Basis: Written for this question (a doubly linked list plus hash map as in LeetCode 146, LRU Cache, with queues bucketed by party size).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
queue = []  # in arrival order
def join(user):
    queue.append(user)
def delete(user):
    queue.remove(user)
def find_first_match(t):
    for user in queue:
        if user.party_size <= t:
            return user
    return None
```

TIME	SPACE
O(n) per find and per delete	O(n)

WHY IT IS SLOW

Delete and find are linear in the list. One queue per party size with lazy deletion makes join and delete O(1) and find touch only the queue fronts.

OPTIMAL SOLUTION · Queue per party size with lazy deletion

TIME	SPACE
O(1) join and delete, O(S) find (S distinct sizes up to the table size)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Restaurant Waitlist: join / delete /
# find_first_match
#
# Onsite coding round: design and implement a restaurant
# waitlist data structure supporting join, delete, and
# find_first_match(table_size). The find call returns the
# earliest-joined user whose party size fits the given
# table. Implement a class with these methods: join(user)
# — append user (with their party_size) to the waitlist.
# delete(user) — remove user from the waitlist (could be
# anywhere in the queue). find_first_match(table_size:
# int) — return the earliest user still on the waitlist
# whose party_size <= table_size. Do not remove on find.
# Implicit constraints Order of joins matters (FIFO within
# those that match). Users may be deleted out of order, so
# a plain array/linked list with O(1) join cannot give
# O(1) delete and find together.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What would you cache to speed up repeated finds?
#
# INTERVIEW NOTES
# Join needs the order of arrival, delete can hit anyone,
# and find must return the earliest person whose party
# fits the table. Keep one queue per party size and a
# sequence number per person: find looks at the front of
# every queue for sizes up to the table size and takes the
# smallest sequence number. Deletions are marked and
# skipped lazily when a queue front is read.
#
# BRUTE FORCE (say it first): Keep one list in join order
# and scan it
# Append on join, remove by value on delete and scan from
# the front on find. Correct and the oracle; delete and
# find are linear. Time O(n) per find and per delete.
# Space O(n).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Keep one list in join order
# and scan it | time O(n) per find and per delete
class WaitlistBrute:
    def __init__(self):
        self.people = []

    def join(self, name, size):
        self.people.append((name, size))

    def delete(self, name):
        self.people = [
            p for p in self.people if p[0] != name
        ]

    def find_first_match(self, table):
        return next(
            (n for n, s in self.people if s <= table),
            None,
        )


# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
        return queue[0] if queue else None

    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
        return best[1] if best else None


if __name__ == "__main__":
    w = Waitlist()
    for name, size in [("Alice", 2), ("Bob", 4)]:
        w.join(name, size)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"
    # find does not remove anyone
    assert w.find_first_match(5) == "Alice"
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    # the same size: arrival order decides
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"
    # deleting someone who is not there is harmless
    w.delete("Nobody")
```

NOTES FROM CANDIDATES

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

PREPARATION

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY PATH: question 134 of 167 (more practice)

Priority score 1.2 | Data structure and object design | asked 297 days ago

MEDIUM | PracHub | Technical Screen | 6 of 7

Match people to questions using tags and priorities

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 172/228

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: the interview site's own solution

LeetCode match: LC 1820 Maximum Number of Accepted Invitations (bipartite matching) (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-06-04)

Original: https://prachub.com/coding-questions/match-people-to-questions-using-tags-and-priorities

QUESTION

Overview: This question evaluates competencies in bipartite matching and combinatorial optimization, along with practical data-structure design for scalable tag-based assignment and priority-aware selection.

APPROACH

Model as bipartite matching: edges where tags overlap, weights by priority; use Kuhn's/Hopcroft-Karp for max matching, or greedy by priority with an inverted index tag -> candidates.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether matching is one-to-one or a question can take many people, what priority means (higher wins, or a numeric weight to maximize), whether a person needs only one shared tag, and whether it must be maximum cardinality or maximum total priority.

BRUTE FORCE TO SAY OUT LOUD

Try every assignment: exponential. State it and move on.

OPTIMAL IDEA

Build an inverted index tag to candidate questions. If only the number of matches matters it is bipartite matching: Kuhn's augmenting-path algorithm in O(V * E) is enough, Hopcroft-Karp for scale. If priorities are weights use min-cost flow or the Hungarian algorithm. If priority is a strict order, process people by priority and augment, because matchings form a matroid and greedy by priority is optimal.

DRY-RUN THESE CASES

A person with no tag overlap. Two people competing for one question. A chain where the first greedy choice blocks a larger matching (shows why plain greedy fails without augmenting).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Capacities per question (turns into max flow), online arrivals, or fairness constraints.

PITFALLS

Assuming plain greedy gives a maximum matching. Not building the index and doing all-pairs tag checks.

HOW TO THINK ABOUT IT

'Assign each person at most one question, each question at most one person, allowed pairs given by tags' is maximum bipartite matching. The tag index only builds the edges faster. For maximum count use augmenting paths: try to give a person a question, and if taken ask its current owner to move. If priorities decide who is served, process people in priority order with the same augmenting step.

Basis: LeetCode 1820, Maximum Number of Accepted Invitations (bipartite matching). Assumed rules (confirm them): a person and a question can be matched if they share a tag; each person gets at most one question and each question at most one person; priority decides who is served first when not everyone can be matched.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 0
def go(person, used, matches):
    global best
    if person == len(people):
        best = max(best, matches)
        return
    go(person + 1, used, matches)  # skip
    for q in range(len(questions)):
        if q in used:
            continue
        if not shares_tag(person, q):
            continue
        go(person + 1, used | {q},
           matches + 1)
```

TIME	SPACE
O((Q + 1)^P)	O(P)

WHY IT IS SLOW

Each person branches over all questions: exponential. Augmenting paths (move an earlier person to another question when needed) solve it in O(people * edges).

OPTIMAL SOLUTION · Augmenting paths (Kuhn's algorithm)

TIME	SPACE
O(P * E)	O(P + Q)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

FOLLOW-UP 2 Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

FOLLOW-UP 3 A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Match people to questions using tags and
# priorities
#
# This question evaluates competencies in bipartite
# matching and combinatorial optimization, along with
# practical data-structure design for scalable tag-based
# assignment and priority-aware selection.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Priorities: higher-priority people must be served
# first.
# 2. Edges have weights (how good a match is).
# 3. A question may take several people (capacity c).
#
# INTERVIEW NOTES
# 'Assign each person at most one question, each question
# at most one person, allowed pairs given by tags' is
# maximum bipartite matching. The tag index only builds
# the edges faster. For maximum count use augmenting
# paths: try to give a person a question, and if taken ask
# its current owner to move. If priorities decide who is
# served, process people in priority order with the same
# augmenting step.
#
# BRUTE FORCE (say it first): Try every assignment
# recursively
# For each person either skip them or give them any unused
# question they share a tag with, and keep the largest
# number of matches. Exact, so it checks the fast
# algorithm, but exponential. Time O((Q + 1)^P). Space
# O(P).


# BRUTE FORCE (say it first): Try every assignment
# recursively | time O((Q + 1)^P)
def max_matching_brute(adj):
    best = 0

    def go(i, used, count):
        nonlocal best
        if i == len(adj):
            best = max(best, count)
            return
        go(i + 1, used, count)
        for q in adj[i]:
            if q not in used:
                go(i + 1, used | {q}, count + 1)

    go(0, frozenset(), 0)
    return best


# BUILD the person-question graph
def build_graph(people, questions):
    # a person can take a question if they share a tag
    # people: [(name, priority, tags)], questions: [(id,
    # tags, cap)]
    adj = {}
    for name, _, tags in people:
        adj[name] = [q for q, qtags, _ in questions
                     if set(tags) & set(qtags)]
    return adj


# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt.
def max_matching(adj):
    # the plain version: adj[p] = list of questions for
    # person p
    owner = {}

    def try_assign(p, seen):
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            free = q not in owner
            if free or try_assign(owner[q], seen):
                owner[q] = p
                return True
        return False

    people = range(len(adj))
    return sum(try_assign(p, set()) for p in people)


# FOLLOW-UP 1: Priorities: higher-priority people must be
# served first.
# Answer: Process people in decreasing priority and run
# the same augmenting step. Matchings of this kind form a
# matroid, so this greedy choice gives a matched set with
# the maximum total priority (checked against exhaustive
# search).
def assign(people, questions):
    # Kuhn's augmenting paths. People are tried by
    # priority
    # (highest first): such matchings form a matroid, so
    # this
    # greedy order keeps the best possible set of people.
    adj = build_graph(people, questions)
    cap = {q: c for q, _, c in questions}
    # question -> the people currently holding its places
    held = {q: [] for q in cap}

    def try_assign(person, seen):
        for q in adj[person]:
            if q in seen:
```

```
                continue
            seen.add(q)
            # a free place, or someone who can move
            # elsewhere
            if len(held[q]) < cap[q]:
                held[q].append(person)
                return True
            for other in list(held[q]):
                if try_assign(other, seen):
                    held[q].remove(other)
                    held[q].append(person)
                    return True
        return False

    order = sorted(people, key=lambda p: -p[1])
    for name, _, _ in order:
        try_assign(name, set())
    return {p: q for q, ps in held.items() for p in ps}


if __name__ == "__main__":
    assert max_matching([[0, 1], [0], [1, 2]]) == 3
    assert max_matching([[0], [0], [0]]) == 1
```

```
    people = [("ann", 1, {"dp"}),
              ("bob", 5, {"dp", "graph"}),
              ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    got = assign(people, questions)
    # bob (priority 5) is served first, and the matching
    # is
    # still the largest possible: ann -> q1, bob or cy ->
    # q2
    assert len(got) == 2
    assert got["bob"] in ("q1", "q2")
    # a question with two places takes two people
    q = [("q1", {"dp"}, 2)]
    both = assign(people[:2], q)
    assert set(both) == {"ann", "bob"}
    # nobody shares a tag: nobody is matched
    nobody = assign([("x", 1, {"a"})], [("q", {"b"}, 1)])
    assert nobody == {}
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[list[str]], question_tags: list[
list[str]], priorities: list[int]) -> list[int]:
```

Sample call

```
solution([['python', 'sql'], ['java'], ['ml']], [['sql'],
['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])
```

Test cases

Input	Expected
`([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])`	`(3, 16)`
`([['a'], ['a']], [['a'], ['a'], ['b']], [1, 10, 100])`	`(2, 11)`

STUDY PATH: question 135 of 167 (more practice)

Priority score 1.0 | Data structure and object design | asked 409 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 7 of 7

Design server allocation for multi-type nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 220/228

ABOUT THIS QUESTION

Topic: More practice - Data structure and object design

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match; related LC 1912 Design Movie Rental System

Posted on PracHub: 2025-09-06 (updated 2026-08-05)

Original: https://prachub.com/interview-questions/design-server-allocation-for-multi-type-nodes

QUESTION

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

APPROACH

Map instance type -> ordered set of free nodes; allocating a node removes it from every type's set (lazy deletion with an allocated flag); release re-adds it. Pick by the rule in the prompt.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what getServer returns when it cannot be satisfied, whether it must be atomic, what collectively meet the types means (every requested type is covered by at least one chosen server), and the scale (about 100,000 nodes).

BRUTE FORCE TO SAY OUT LOUD

Scan all nodes on every request and test their types. O(N) per call.

OPTIMAL IDEA

Keep, for each instance type, a heap or sorted set of free node ids ordered by how few types the node supports (least flexible first). To serve, pick for each requested type the least flexible free node not already chosen, then fill the remaining count with least flexible nodes that match any requested type. Mark nodes allocated (lazy deletion in the other types' structures). Roll back if the request cannot be completed. Release re-adds the node to each of its types.

DRY-RUN THESE CASES

Request larger than the free pool, a node shared by two requested types, return then re-request, releasing an id that is not allocated.

LIKELY FOLLOW-UPS

Concurrency (per-type locks or a single allocator thread), starvation (FIFO queue), fragmentation, failure handling (leases and timeouts).

PITFALLS

Allocating a node and forgetting to remove it from the other type indexes. Greedy choices that are not optimal set cover; say so.

INTERVIEW-DAY PLAN

Keep, for each instance type, a heap or sorted set of free node ids ordered by how few types the node supports (least flexible first). To serve, pick for each requested type the least flexible free node not already chosen, then fill the remaining count with least flexible nodes that match any requested type. Mark nodes allocated (lazy deletion in the other types' structures). Roll back if the request cannot be completed. Release re-adds the node to each of its types.

DRY-RUN THESE CASES

Request larger than the free pool, a node shared by two requested types, return then re-request, releasing an id that is not allocated.

LIKELY FOLLOW-UPS

Concurrency (per-type locks or a single allocator thread), starvation (FIFO queue), fragmentation, failure handling (leases and timeouts).

PITFALLS

Allocating a node and forgetting to remove it from the other type indexes. Greedy choices that are not optimal set cover; say so.

HOW TO THINK ABOUT IT

A request mentions a few types, so index the free servers by type and avoid scanning all 100,000. To keep versatile servers available for harder requests, always prefer the server that supports the fewest types. Allocation touches every type index of a server, so use lazy deletion: mark it allocated and let the stale heap entries be skipped when they surface.

Basis: Written design. Assumed contract (confirm it): getServer(count, types) returns `count` free server ids such that every requested type is supported by at least one returned server, or None and no change; releaseServer(ids) frees them. To limit fragmentation the least flexible servers (supporting the fewest types) are used first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def get_server(count, types):
    chosen = []
    for t in types:
        pool = [s for s in free if t in s.types
                and s not in chosen]
        chosen.append(min(pool, key=flexibility))
    rest = [s for s in free if s not in chosen
            and s.types & set(types)]
    chosen += rest[:count - len(chosen)]
    if len(chosen) < count:
        return None  # not enough
    return chosen
```

TIME	SPACE
O(N * T)	O(N)
per request (N servers)	

WHY IT IS SLOW

BRUTE FORCE

Every request scans all N servers, O(N * types). Per-type heaps ordered by flexibility with lazy deletion make a request O((count + types) log N).

OPTIMAL SOLUTION · Per-type heaps ordered by flexibility, with lazy deletion

TIME	SPACE
O((count + types) log N) amortized per request	**O(total type memberships)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

FOLLOW-UP 2 Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

FOLLOW-UP 3 Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

FOLLOW-UP 4 A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design server allocation for multi-type
# nodes
#
# Design a service to manage a large pool (about 100, 000)
# of server nodes. Each node can satisfy multiple instance
# types; clients call getServer(reqNumOfServer,
# instanceTypes[]) to request servers that collectively
# meet the requested types, and returnServer(ids[]) to
# release them. Propose data structures and indexes for
# fast matching, availability tracking, fragmentation
# minimization, and scaling. Address concurrency,
# starvation, and failure handling. Overview: This
# question evaluates understanding of scalable resource-
# allocation and matching systems, encompassing data-
# structure and index design, availability tracking,
# fragmentation minimization, concurrency control, and
# failure handling for large node pools.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many clients call at the same time.
# 2. Clients keep asking for rare types and starve.
# 3. Servers fail while allocated.
# 4. A client dies without releasing its servers.
#
# INTERVIEW NOTES
# A request mentions a few types, so index the free
# servers by type and avoid scanning all 100,000. To keep
# versatile servers available for harder requests, always
# prefer the server that supports the fewest types.
# Allocation touches every type index of a server, so use
# lazy deletion: mark it allocated and let the stale heap
# entries be skipped when they surface.
#
# BRUTE FORCE (say it first): Scan every free server on
# each request
# For each requested type find the best free server by
# scanning all servers, then fill the remaining count the
# same way. Simple and a good oracle; every request
# touches every server. Time O(N * T) per request (N
# servers). Space O(N).


import heapq
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Scan every free server on
# each request | time O(N * T) per request (N servers)
class ServersBrute:
    def __init__(self, servers):
        self.types = dict(servers)
        self.free = set(servers)

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if any(t in self.types[c] for c in chosen):
                continue
            cands = [
                s
                for s in self.free
                if s not in chosen
                and t in self.types[s]
            ]
            if not cands:
                return None
            chosen.append(
                min(
                    cands,
                    key=lambda s: (
                        len(self.types[s]),
                        s,
                    ),
                )
            )
        extra = sorted(
            (
                s
                for s in self.free
                if s not in chosen
                and self.types[s] & set(types)
            ),
```

```
            key=lambda s: (len(self.types[s]), s),
        )
        if (
            len(chosen) > count
            or len(chosen) + len(extra) < count
        ):
            return None
        chosen += extra[: count - len(chosen)]
```

```
        self.free -= set(chosen)
        return chosen

    def releaseServer(self, ids):
        self.free |= set(ids)


# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it.
# FOLLOW-UP 4: A client dies without releasing its
# servers.
# Answer: Give every allocation a lease (expiry time). A
# periodic sweep returns servers whose lease has expired
# to the free pool (the lazy-deletion indexes make a
# release cheap), and clients renew the lease while they
# are alive.
class ServerPool:
    """Servers with several instance types. Each type has
    a heap of (number of types, id): the LEAST flexible
    server goes first, which keeps versatile ones free."""

    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {s: frozenset(t)
                      for s, t in servers.items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        # FOLLOW-UP 4: a dead client's lease expires
        # lease expiry time of every allocated server
        self.lease = {}
        for s, ts in self.types.items():
            for t in ts:
                heap = self.heaps[t]
                heapq.heappush(heap, (len(ts), s))

    def best_for(self, t, skip):
        # the best free server of type t that is not in
        # `skip`;
        # allocated entries are dropped lazily
        heap, held = self.heaps[t], []
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
```

```
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
            else:
                break
        best = heap[0] if heap else None
        # put back the entries we only looked past
        for item in held:
            heapq.heappush(heap, item)
        return best

    def get_server(self, count, types):
        # pick servers first, commit only if the whole
        # request works
        chosen = []
        for t in types:
            # a server already chosen may cover this type
            if any(t in self.types[c] for c in chosen):
```

```
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        # fill up to `count` with the least flexible free
        # servers
        while len(chosen) < count:
            skip = set(chosen)
            found = [self.best_for(t, skip)
                     for t in types]
            options = [o for o in found if o]
            if not options:
                return None
            chosen.append(min(options)[1])
        # more servers than allowed were needed to cover
        # the types
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def release(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                self.lease.pop(s, None)
```

```
                flex = len(self.types[s])
                for t in self.types[s]:
                    heap = self.heaps[t]
                    heapq.heappush(heap, (flex, s))

    def rent(self, count, types, now, ttl):
        # a client that dies must not hold servers forever
        got = self.get_server(count, types)
        for s in got or []:
            self.lease[s] = now + ttl
        return got

    def expire(self, now):
        # return servers whose lease ran out to the free
        # pool
        late = [s for s in self.lease
                if self.lease[s] <= now]
        self.release(late)
        return late


if __name__ == "__main__":
    pool = ServerPool({1: {"gpu"}, 2: {"gpu", "big"},
                       3: {"big"}, 4: {"cpu"}})
    # the least flexible server for each type is used
    # first
    assert pool.get_server(2, ["gpu", "big"]) == [1, 3]
    # the versatile server 2 is still there for a combined
    # need
    assert pool.get_server(1, ["gpu", "big"]) == [2]
    # nothing is left for a gpu request
    assert pool.get_server(1, ["gpu"]) is None
    # a request that cannot fit in `count` servers changes
    # nothing
    assert pool.get_server(1, ["cpu", "gpu"]) is None
    assert pool.get_server(1, ["cpu"]) == [4]
    pool.release([2])
    assert pool.get_server(1, ["gpu"]) == [2]
    # leases: a dead client's servers come back
    p2 = ServerPool({1: {"a"}})
    assert p2.rent(1, ["a"], now=0, ttl=10) == [1]
    assert p2.get_server(1, ["a"]) is None
    assert p2.expire(now=5) == []
    assert p2.expire(now=10) == [1]
    assert p2.get_server(1, ["a"]) == [1]
```